

# Python Object-Oriented Programming

## 1. Classes and Objects

A class is a blueprint for creating objects. It defines attributes and methods that describe the properties and behavior of objects. An object is an instance of a class and represents a concrete entity created from that blueprint.

## 2. Encapsulation

Encapsulation is the practice of combining data and the methods that operate on that data within a class. It also helps control access to an object's internal state.

## 3. Inheritance

Inheritance allows a child class to acquire attributes and methods from a parent class. It supports code reuse and allows a new class to extend or specialize the behavior of an existing class. For example, a class named Car could inherit common properties and methods from a class named Vehicle.

## 4. Polymorphism

Polymorphism means that the same interface or method name can be used with different types of objects, while each object can provide its own implementation of the behavior. It allows programs to work with objects in a flexible way.

## 5. Abstraction

Abstraction focuses on exposing the essential features of an object while hiding unnecessary implementation details. It helps reduce complexity and makes software easier to use and maintain.

## 6. Relationship Between the Concepts

Classes define the structure of objects. Encapsulation organizes and protects data and behavior. Inheritance enables reuse between related classes. Polymorphism allows different objects to respond to a common interface in different ways. Abstraction hides unnecessary implementation details.